```
Created new squashed migration
  0006_promoter_birth_squashed_0007_promoter_death.py

  You should commit this migration but leave the old ones in place;
  the new migration will be used for new installs. Once you are sure
  all instances of the codebase have applied the migrations you squashed,
  you can delete them.
```

The command asks if you are sure and results in a new file that starts with the same number as the source of your squashing. The new state of your migration directory is as follows:

```
RiffMates/promoters/migrations/
├── 0001_initial.py
├── 0002_promoter_famous_for.py
├── 0003_promoter_common_name_promoter_full_name.py
├── 0004_data_fullname.py
├── 0005_remove_promoter_first_name_remove_promoter_last_name.py
├── 0006_promoter_birth.py
├── 0006_promoter_birth_squashed_0007_promoter_death.py
├── 0007_promoter_death.py
└── __init__.py
```

Original script for adding the .birth field

Squashed script containing the addition of both the .birth and .death fields

Original script for adding the .death field

As the note from the command tells you, you can leave the old files around. The Django migration framework ignores them. Of course, if you were willing to keep them around, there really wasn't a need to squash them in the first place, but if you want to keep them there until you're sure everything works, they do no damage. The more complicated your migration files are, the more work Django has to do to squash your migrations. There is a chance the tool can get it wrong, and the documentation recommends holding onto your old files until you've tested the squashed ones. See https://mng.bz/7ddx for more information and to better understand possible pitfalls.

10.4 More migration choices

So far, you've mostly been running the `migrate` management command without any options. When you call `migrate` without any arguments, it migrates the database forward to the current state of your model files. You can be more explicit, though, and specify the target state. Doing this, you can migrate forward or backward to any allowed position on the migration timeline.

For example, the current state of `Promoter` is the squashed migration script with the birth and death field additions. To back this migration out, you can migrate to stage 0005:

Specify both the Django app name and (partial) migration filename to target a specific state.

```
(venv) RiffMates$ python manage.py migrate promoters 0005
Operations to perform:
  Target specific migration:
    0005_remove_promoter_first_name_remove_promoter_last_name, from promoters
```